

Inigo Quilez :: articles :: sphere visibility - 2008



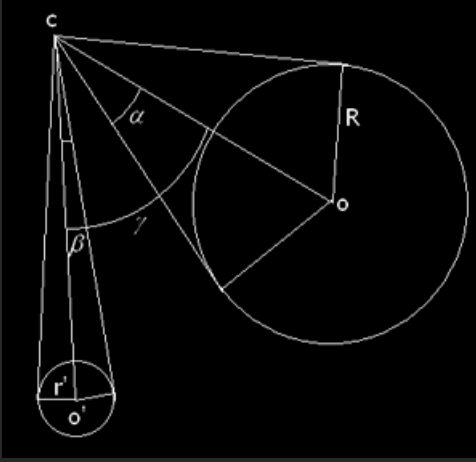
Motivation

It seems that I'm working a lot on spheres recently... or that someone close to me does so ;) Well, thing is I recently got some new equations for a neat trick that you can add to your toolbox too. Say you have a planet, or in other words, a sphere (where you most probably grow some procedural mountains or whatever). Now, say you have objects both at planetary surface level as well as in the sky or space. Given a camera position (both in surface or space), you want to know what objects are for sure NOT visible because of planet occlusion. Objects in the other side of the planet will not be visible of course, but many others will also not be because of the curvature of the planet, and that can count for a BIG amount of geometry when you are at surface level ("objects" includes "mountains" also of course).

The trick presented here can be seen at a kind of horizon mapping, and it gives analytical micropixel perfect occlusion results, for a spherical occluder(the planet) and spherical objects. Now, you can use spheres around every node of your octree to quickly discard complete subtrees of geometry on your planet :)

The maths

We start with the following construction (shown on the left of this text). Let $\mathbf{c}$ be the camera position (we assume it's not inside any of the two spheres). The big sphere with center position $\mathbf{o}$ and radius R is the occluder (our planet). The small sphere with at position $\mathbf{o}'$ and radius R' is the bounding sphere of the object that we are testing occlusion for.



The key observation is that the small sphere will be visible or partially visible as long as

$$\gamma > \alpha + \beta$$

Of course this doesn't take into account the case where the little sphere is between the camera and the planet, but that case we will handle separately later on. Working with angles is always a bad idea, not only it involves expensive inverse trigonometric functions, but also they are error prone because of angle value wrappings at 2π and so on. It's much more desirable to work just with vectors (vectors never lie!). So, we convert the previous condition by taking cosines in both sides (cosines are always dot products of vectors!):

$$\cos(\gamma) < \cos(\alpha + \beta) = \cos(\alpha) \cdot \cos(\beta) - \sin(\alpha) \cdot \sin(\beta)$$

Note that the direction of the comparison changed of course. Now we have to identify all these sines and cosines. Looking to the image above, we get:

$$\begin{aligned} \sin \alpha &= R / D & \cos \alpha &= \frac{\sqrt{D^2 - R^2}}{D} \\ \sin \beta &= R' / D' & \cos \beta &= \frac{\sqrt{D'^2 - R'^2}}{D'} \end{aligned}$$

where

$$D = |o - c| \quad D' = |o' - c|$$

Let's call $k = \cos(\gamma)$ and put it all together to get:

$$kDD' - RR' < \sqrt{(R^2 - D^2)(R'^2 - D'^2)}$$

This is much nicer already than the angle nightmare. However we can do better and get rid of the square root. Note that we want to flag objects as visible when the inequality holds. If the left part of the inequality is negative, the object will be this visible for sure. If it's positive, we have to compute the complete thing. Before going on, let's introduce

$$k_1 = \frac{R}{D} \quad k_2 = \frac{R'}{D'}$$

and see that we can check for the sign of the left part by

$$k > k_1 \cdot k_2$$

Now, the complete equation can be simplified by taking squares in both sides. The resulting inequality is:

$$k_1^2 + k_2^2 < 1 - k^2 + 2k \cdot k_1 \cdot k_2$$

what is quite cool (simple and fast to compute, ie, elegant!) You can reinterpret it as

$$\sin^2 \alpha + \sin^2 \beta < \sin^2 \gamma + 2 \cos \gamma \sin \alpha \sin \beta$$

what looks like the a kind of cosine law btw :)

Implementation

The implementation is straightforward. However, one can realize that we can get rid of the two inverse square roots and have a totally divisions and square root free! That makes it extremelly efficient:

```
int sphereVisibility( in vec3 ca, in float ra, in vec3 cb, float rb, in vec3 c )
{
    float aa = dot(ca-c,ca-c);
    float bb = dot(cb-c,cb-c);
    float ab = dot(ca-c,cb-c);
```

```
float s = ab*ab + ra*ra*bb + rb*rb*aa - aa*bb;  
float t = 2.0*ab*ra*rb;  
  
if( s + t < 0.0 ) return 1;  
else if( s - t < 0.0 ) return 2;  
return 3;  
}
```

Note that this method is a kind of 2D overlap test, it will report occlusion when the object is the "shadow" of the planet but still in front of it. The case can be easily fixed by adding a plane test. The plane is formed by the circular cap where the cone formed by the planet the the camera (being the later the apex) intersects the sphere (a disk). The center of the disk can be computed as in [this article](#), and then a dot product will tell you if your object is between that plane and the camera.

A realtime interactive implementation of the code above can be found here (click in the title to navigate to the source code, or simply move the mouse along the image to change the position of the spheres. You can find code here: <https://www.shadertoy.com/view/XdS3Rt>.



inigo quilez - learning computer graphics since 1994